Layout Widgets in Flutter and State Management

Experiment 2(b): Row, Column, Stack
Experiment 5(b): setState() and Provider

UI DESIGN FLUTTER | CSE / IT Laboratory

IMAGE WIDGET

To display any kind of image.

The image can be of different types:

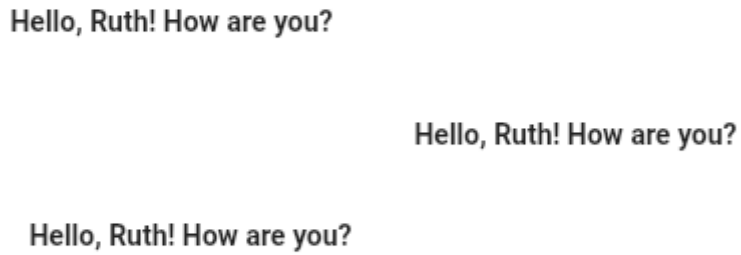
- Asset Image (stored on computer)
- Network image (can be accessed via link)

Etc..

Image.asset() in flutter code
or
Image.network() in flutter code

TEXT WIDGET

A text widget helps to display text on mobile screen



Text("Hello, Ruth! How are you?")

CONTAINER WIDGET


```
Container(
      child: Text("Container 1")
),

Container(
      child: Text("Container 2")
),
```

Experiment 2(b) – Layout Widgets

Implement different layout structures using Row, Column and Stack

- Flutter layouts are built by arranging widgets inside parent widgets.
- Row arranges children horizontally.
- Column arranges children vertically.
- Stack places children on top of one another.
- These widgets can be combined to create complete user interfaces.

ROW WIDGET

A row widget is like a widget in X-Plane. (X Axis)

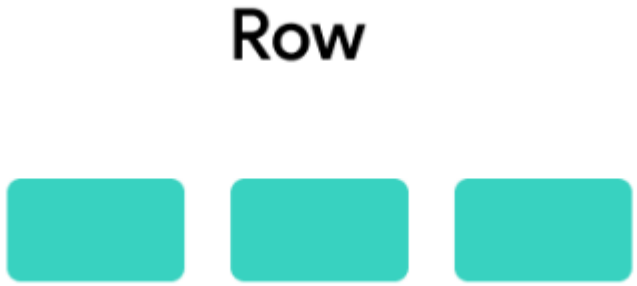
A row can have multiple children widgets

```
Row(
    children:
    [
        Icon(),
        Icon(),
        Icon(),
        Spacer(),
        Icon(),
    ]
),
```

Row Widget

Row places its child widgets from left to right.

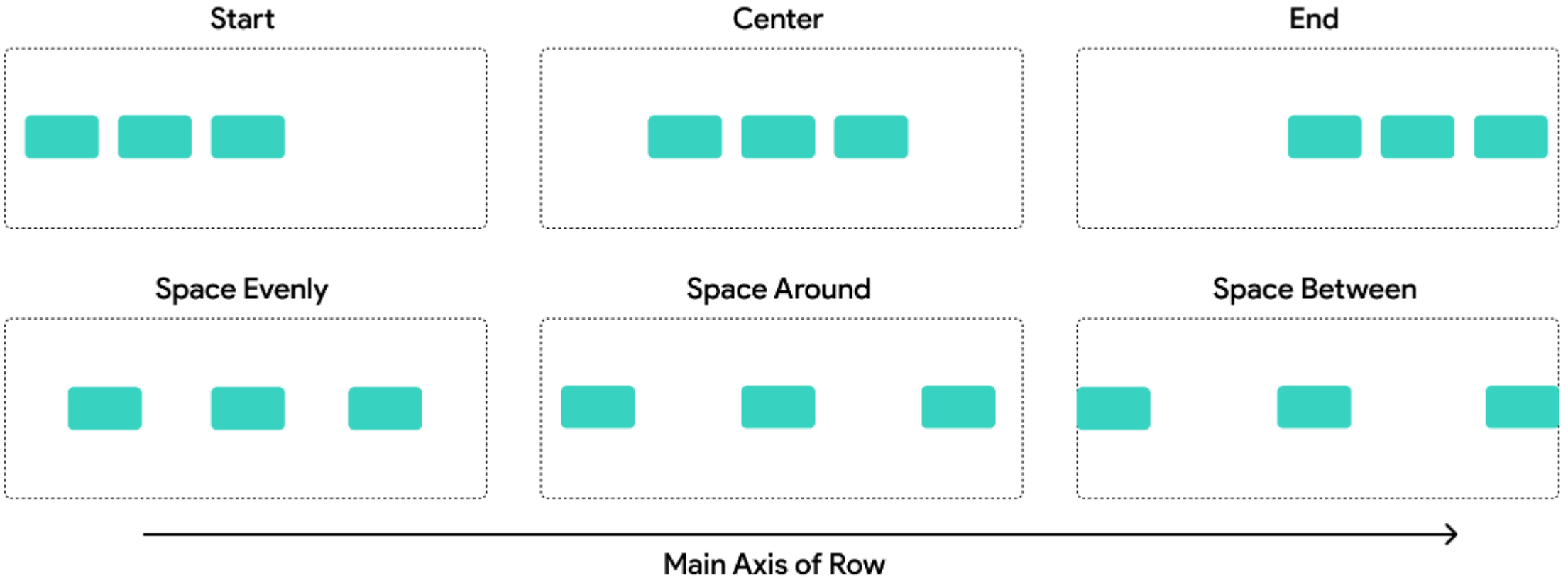
Useful for menus, icon + text combinations, buttons, and horizontal sections.



Important properties:

- children – list of widgets arranged horizontally
- mainAxisAlignment – controls horizontal distribution
- crossAxisAlignment – controls vertical alignment

The main axis is the primary direction, child widgets are laid out in a Row or Column.



Row's Main Axis property has the following types: Start, End, Center, SpaceEvenly, SpaceAround, SpaceBetween

mainAxisAlignment Options

Value	Behavior
start	Children align to the left
center	Center horizontally
end	Align to right
spaceBetween	Space distributed between children
spaceAround	Equal space around children
spaceEvenly	All spacing equal

crossAxisAlignment Options

Value	Behavior
start	Align top
center	Center vertically
end	Align bottom
stretch	Children expand vertically (if possible)

Main Axis vs Cross Axis

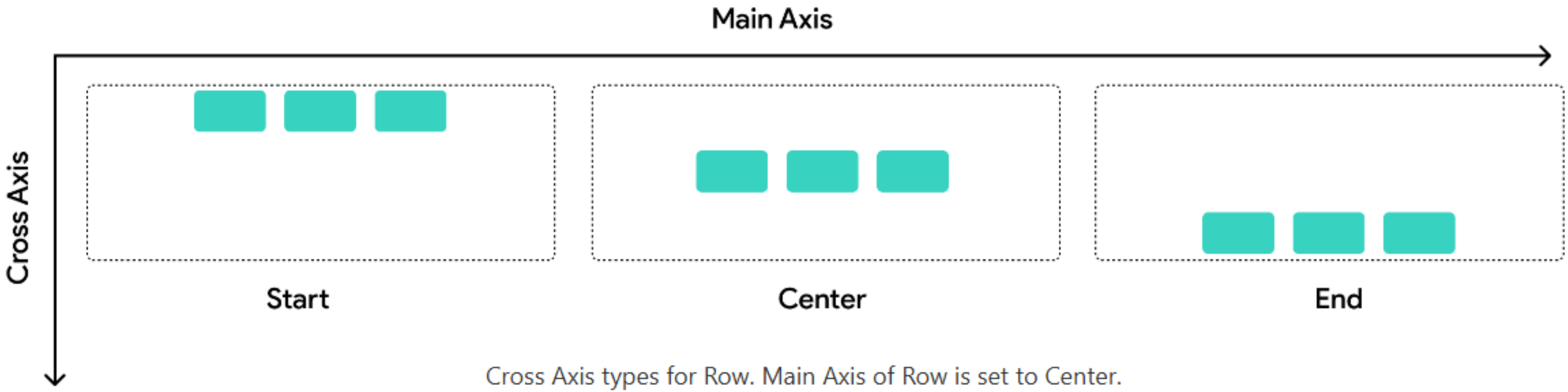
Main Axis → Vertical

Controls how children are aligned top-to-bottom.

Cross Axis → Horizontal

Controls how children are aligned left-to-right.

Row: The cross axis runs **vertically**. It determines how child widgets are aligned from top to bottom within the row.



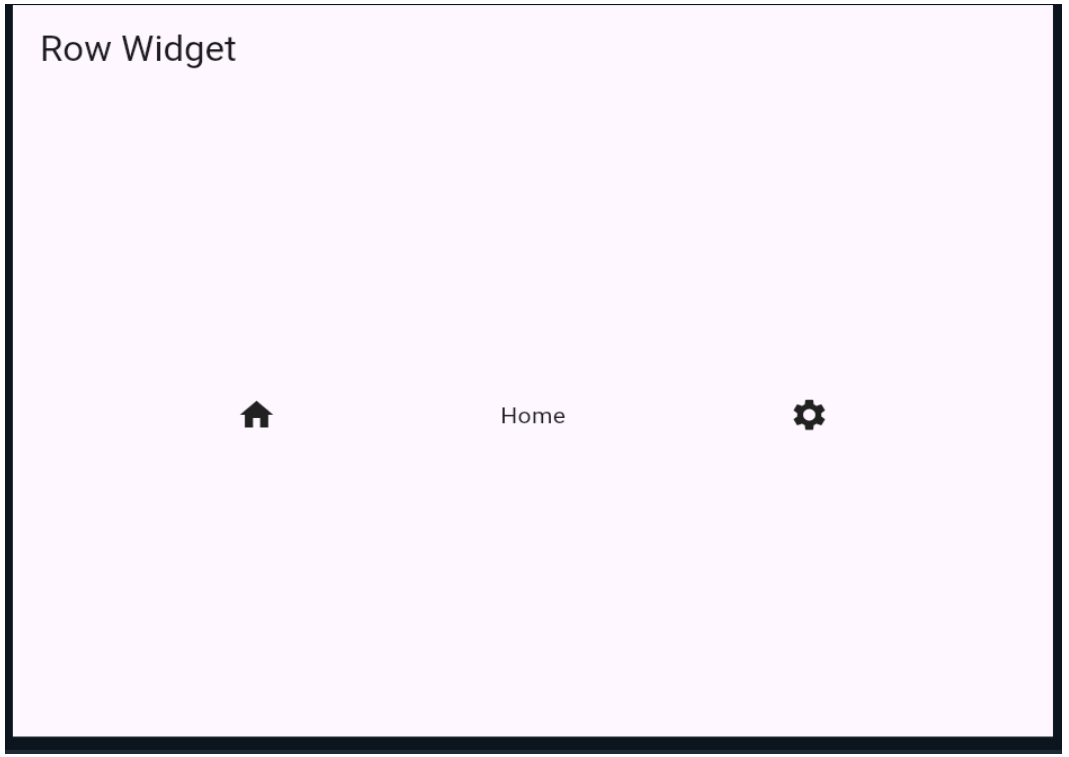
Cross Axis types for Row. Main Axis of Row is set to Center.

Row – Example

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: [
    Icon(Icons.home),
    Text('Home'),
    Icon(Icons.settings),
  ],
)
```

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Row Widget'),
        ),
```

```
 body: Center(
          child: Row(
            mainAxisAlignment:
MainAxisAlignment.spaceEvenly,
            children: const [
              Icon(Icons.home),
              Text('Home'),
              Icon(Icons.settings),
            ],
          ),
        ),
      ),
    );
  }
}
```



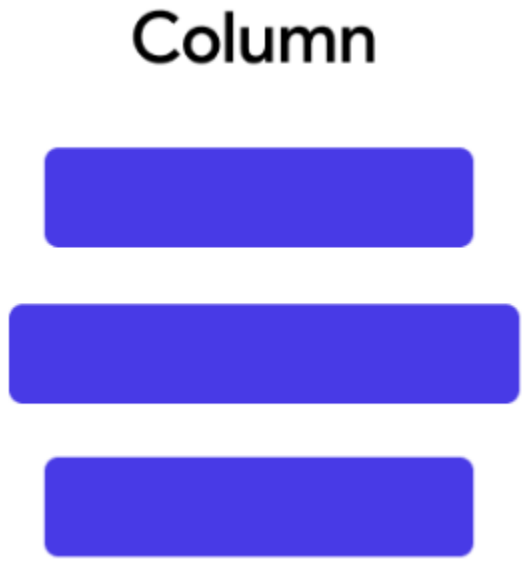
Column Widget

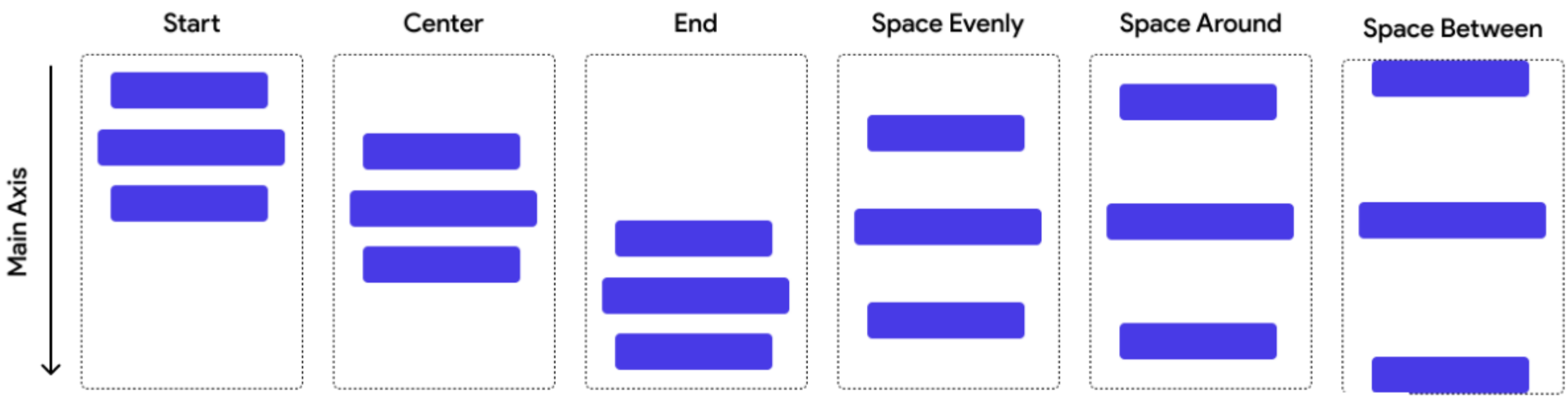
Column places its child widgets from top to bottom.

Useful for forms, login pages, profile screens, and vertical menus.

Important properties:

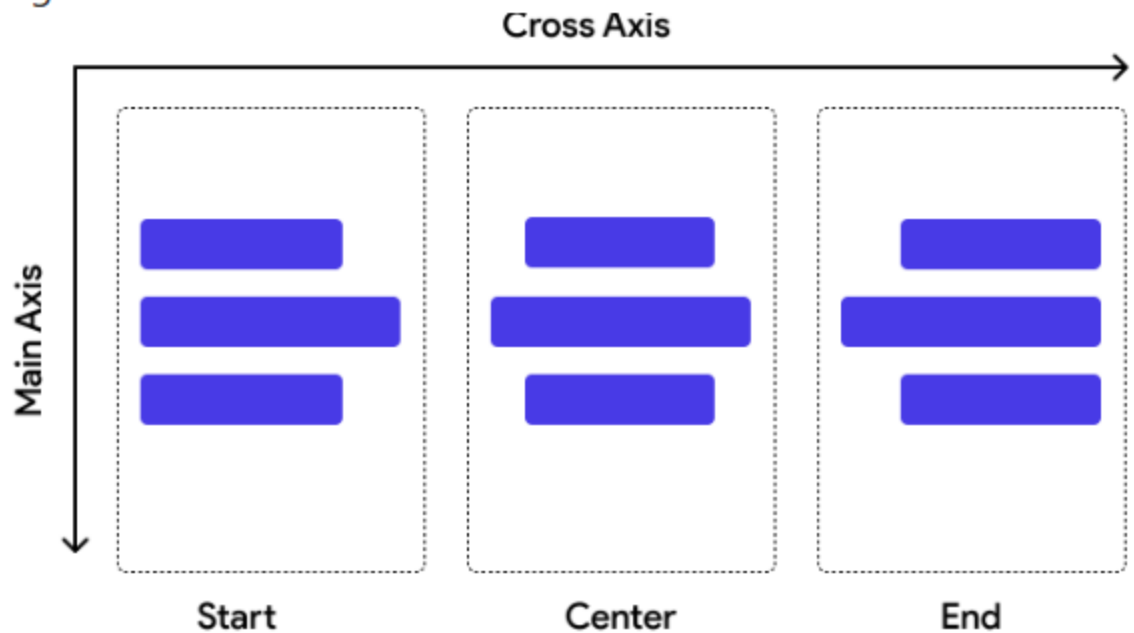
- children – list of widgets arranged vertically
- mainAxisAlignment – controls vertical distribution
- crossAxisAlignment – controls horizontal alignment





Column's Main Axis property has the following types: Start, End, Center, SpaceEvenly, SpaceAround, SpaceBetween

Column: The cross axis runs **horizontally**. It controls how child widgets align from left to right within the column.

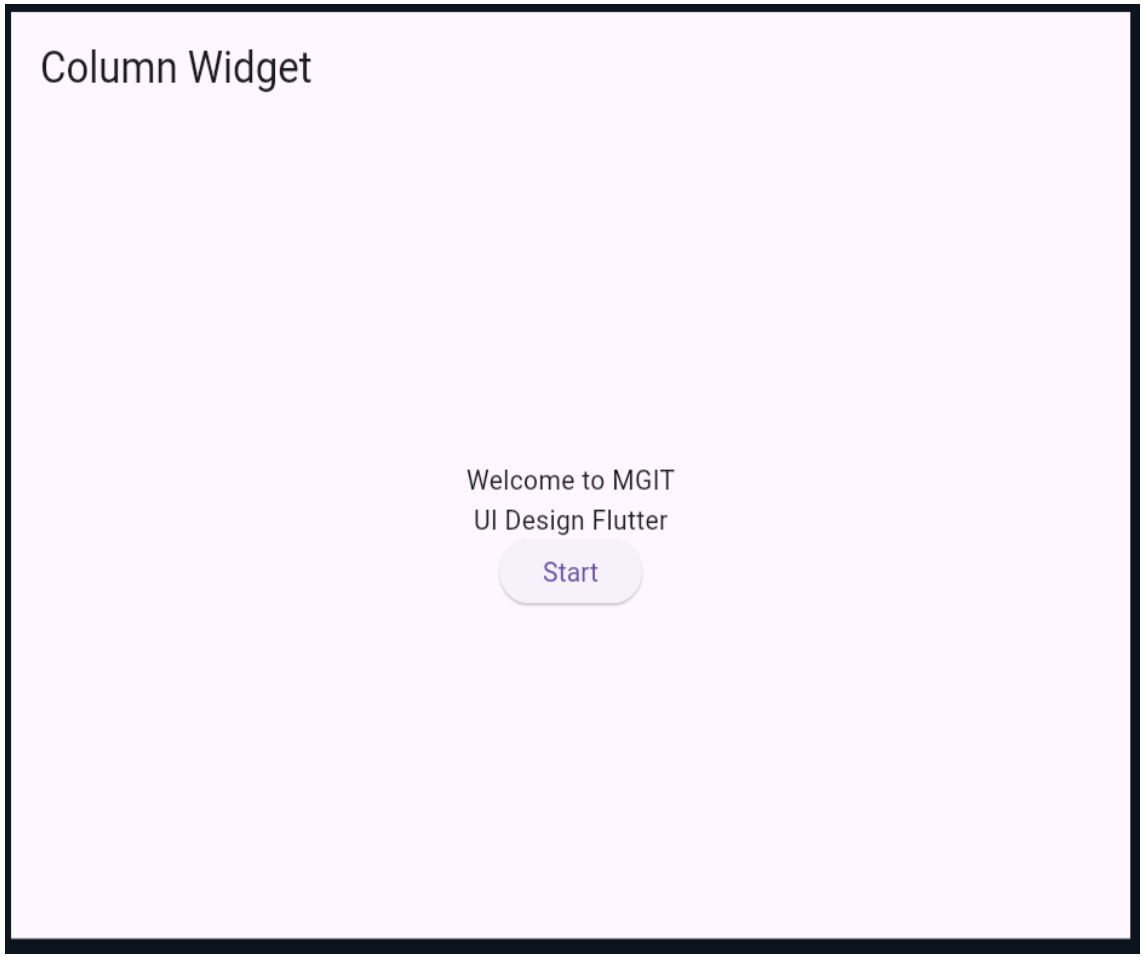


Cross Axis types for Column. Main Axis of Column is set to Center

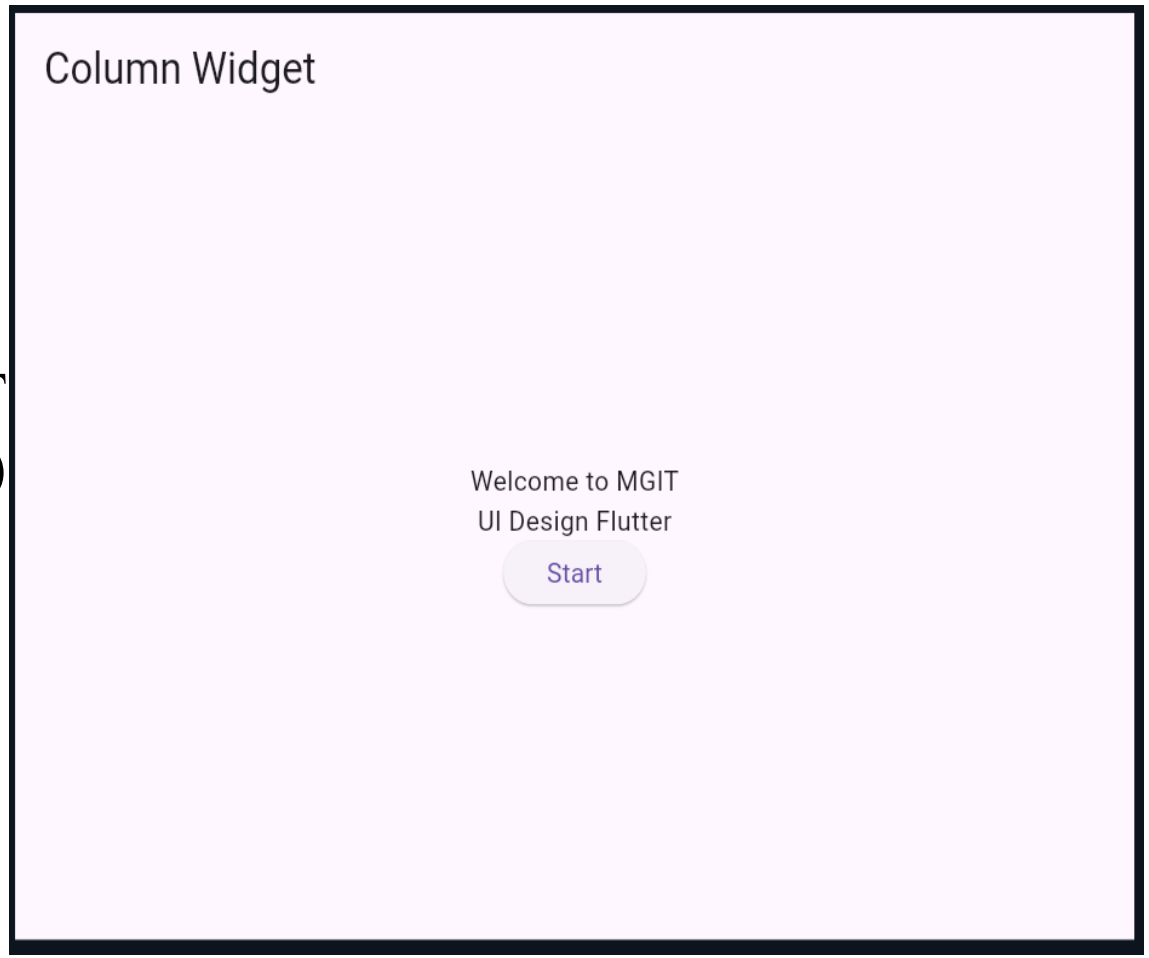
Column – Example

```
Column(
  mainAxisAlignment: MainAxisAlignment.center,
  crossAxisAlignment: CrossAxisAlignment.center,
  children: [
    Text('Welcome to MGIT'),
    Text('UI Design Flutter'),
    ElevatedButton(
      onPressed: () {},
      child: Text('Start'),
    ),
  ],
)
```

```
import 'package:flutter/material.dart';
void main() {
  runApp(const MyApp());
}
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Column Widget'),
        ),
```



```
 body: Center(
        child: Column(
          mainAxisAlignment:
MainAxisAlignment.center,
          crossAxisAlignment:
CrossAxisAlignment.center,
          children: [
            const Text('Welcome to MGIT
            const Text('UI Design Flutter')
            ElevatedButton(
              onPressed: () {
                print('Start button clicked');
              },
              child: const Text('Start'),
            ),
          ],
        ),
      ),
    )
```



MainAxisAlignment and CrossAxisAlignment

mainAxisAlignment controls alignment along the widget's main direction.

For Row: main axis = horizontal.

For Column: main axis = vertical.

crossAxisAlignment controls alignment perpendicular to the main axis.

Common values: start, center, end, spaceBetween, spaceAround, spaceEvenly.

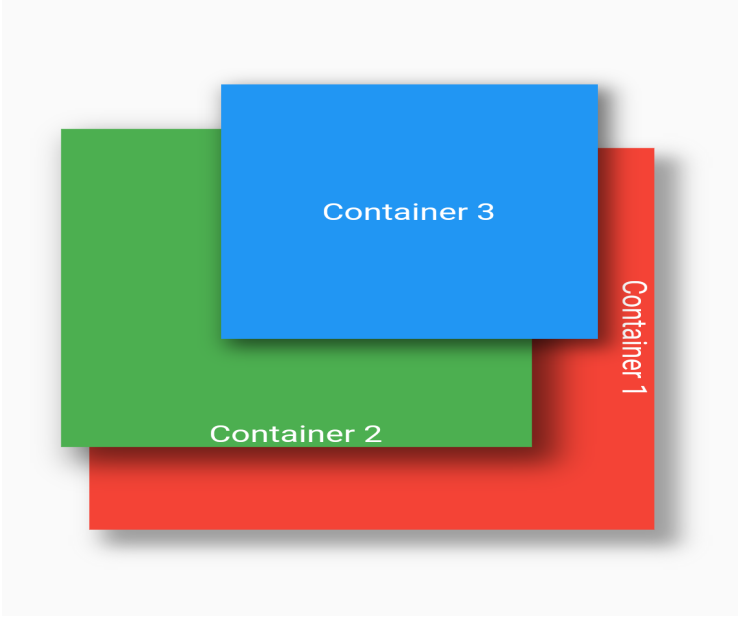
Stack Widget

Stack places widgets one above another.

The first child is at the back; later children appear above it.

Useful for image overlays, badges, profile pictures and floating labels.

Positioned can be used to control the location of a child inside a Stack.



Stack – Example

```
Stack(
  children: [
    Image.network('image_url'),
    Positioned(
      bottom: 10,
      left: 10,
      child: Text(
        'MGIT',
        style: TextStyle(color: Colors.white),
      ),
    ),
  ],
)
```

```
import 'package:flutter/material.dart';
void main() {
  runApp(const MyApp());
}
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Stack Widget'),
        ),
        body: Center(
          child: Stack(
            children: [
              Image.network(
                'https://picsum.photos/300/200',
                width: 300,
                height: 200,
                fit: BoxFit.cover,
              ),
const Positioned(
                bottom: 10,
                left: 10,
                child: Text(
                  'MGIT',
                  style: TextStyle(
                    color: Colors.white,
                    fontSize: 24,
                    fontWeight:
FontWeight.bold,
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
```

Stack Widget

DEBUG

Combined Layout – Row + Column + Stack

Real applications normally combine multiple layout widgets.

Example structure:

- Column – main vertical structure
- Row – horizontal group of actions
- Stack – image with text/icon overlay

Practice: design a simple student profile screen using all three.

```
import 'package:flutter/
material.dart';

void main() {
  runApp(const MyApp());
}
class MyApp extends
StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context)
{
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Combined
Layout'),
        ),
        body: Column(
          children: [
```

```
// Stack: Text over Image
          Stack(
            children: [
              Image.network(
                'https://picsum.photos/400/200',
                width: double.infinity,
                height: 200,
                fit: BoxFit.cover,
              ),
              const Positioned(
                bottom: 10,
                left: 10,
                child: Text(
                  'MGIT',
                  style: TextStyle(
                    color: Colors.white,
                    fontSize: 24,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ),
            ],
          ),
```

```
// Column: Text arranged vertically
        const Column(
          children: [
            Text(
              'Welcome to MGIT',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
            Text('UI Design using Flutter'),
          ],
        ),
const SizedBox(height: 20),

            // Row: Widgets arranged horizontally
            Row(
              mainAxisAlignment:
MainAxisAlignment.spaceEvenly,
              children: [
                const Icon(Icons.home),
                const Icon(Icons.school),
                ElevatedButton(
                  onPressed: () {},
                  child: const Text('Start'),
                ),
              ],
            ),
          ],
        ),
```

Combined Layout

DEBUG

Welcome to MGIT

UI Design using Flutter

Start

Experiment 2(b) – Lab Exercise

Create a student profile UI.

Use Column for the main screen.

Use Stack for profile image + status/edit icon.

Use Row for name, department and action buttons.

Apply alignment properties and spacing.

Run the application and verify the layout on the emulator.

Experiment 5(b) – State Management

Implement state management using setState and Provider

State is data that can change while an application is running.

Examples: counter value, selected item, login status, form values.

When state changes, the UI may need to be rebuilt.

Flutter provides setState() for local state and Provider for shared/application state.

setState()

setState() is used inside a StatefulWidget's State class.

It tells Flutter that the state has changed and the affected UI should rebuild.

The state-changing code is placed inside setState(() { ... }).

Best suited for simple, local state.

Counter using setState()

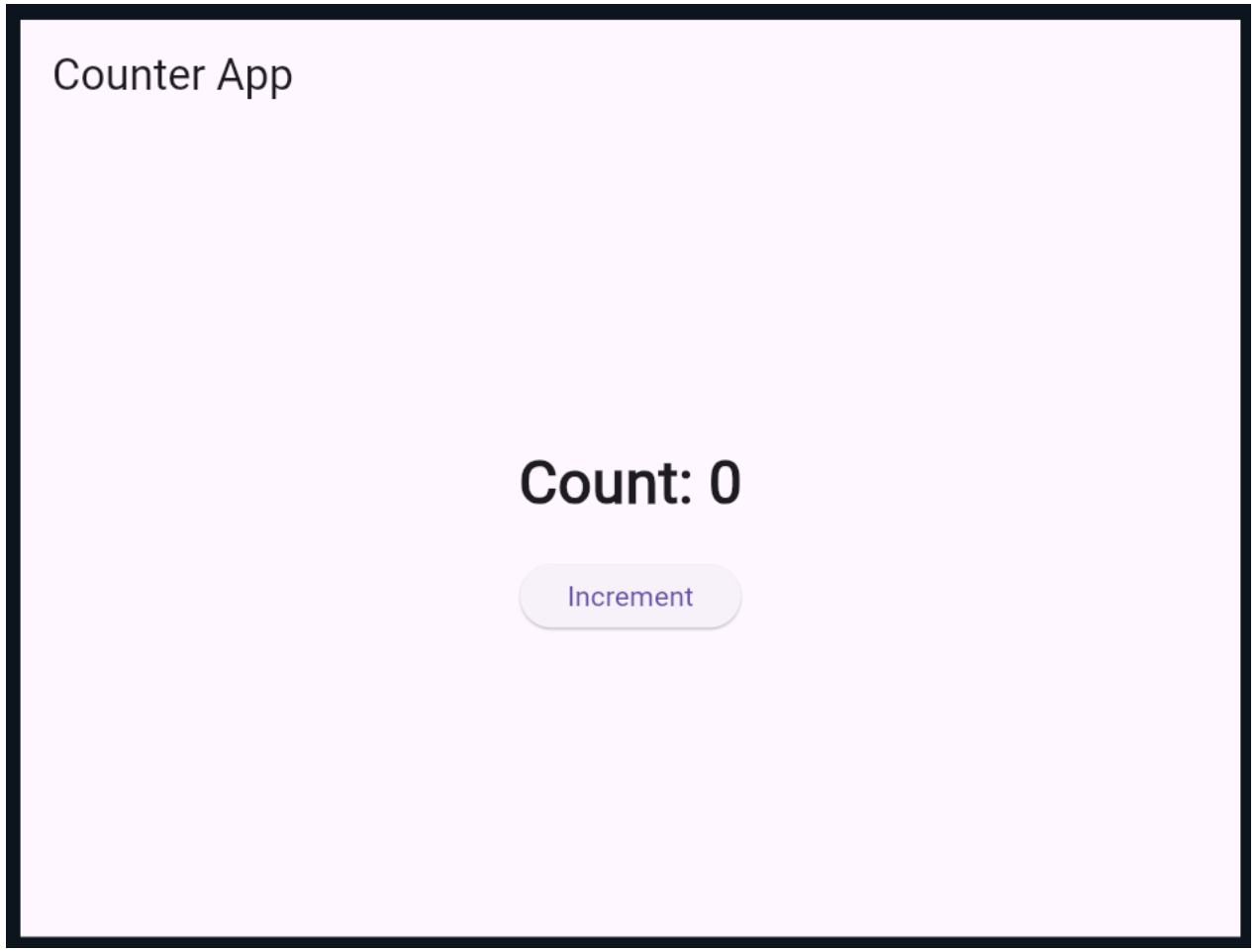
```
class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int count = 0;

@override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text('Count: $count'),
            ElevatedButton(
              onPressed: () {
                setState(() {
```

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: const CounterPage(),
    );
  }
}
```

```dart
class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() =>
_CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int count = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Counter App'),
      ),
```

```
body: Center(
        child: Column(
          mainAxisAlignment:
MainAxisAlignment.center,
          children: [
            Text(
              'Count: $count',
              style: const TextStyle(
                fontSize: 30,
                fontWeight: FontWeight.bold,
              ),
            ),
```

```
const SizedBox(height: 20),

            ElevatedButton(
              onPressed: () {
                setState(() {
                  count++;
                });
              },
              child: const Text('Increment'),
            ),
          ],
        ),
      ),
    );
  }
}
```


Counter App
Count: 0
Increment

How setState() Works

1. User performs an action.
2. Event handler changes the state.
3. setState() notifies Flutter that the state changed.
4. Flutter rebuilds the relevant widget.
5. Updated value appears on the screen.

Flow: User Action → State Change → setState() → Rebuild UI

Why Provider?

setState() is convenient for small, local state.

As applications grow, state may be needed by multiple widgets.

Provider helps expose shared state to widgets without passing values through many widget constructors.

Provider commonly works with ChangeNotifier and notifyListeners().

Provider – Main Concepts

ChangeNotifier – class that holds changeable application state.

notifyListeners() – informs listening widgets that state changed.

ChangeNotifierProvider – makes the state available to descendant widgets.

Consumer – rebuilds a widget when the provided state changes.

Provider.of<T>(context) – accesses the provided object.

Provider – Counter Model

```
import 'package:flutter/material.dart';

class CounterModel extends ChangeNotifier {
  int count = 0;

  void increment() {
    count++;
    notifyListeners();
  }
}
```

Provider – Supplying the State

```
ChangeNotifierProvider(
  create: (_) => CounterModel(),
  child: MyApp(),
)

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: CounterPage(),
    );
  }
}
```

Provider – Reading the State

```
Consumer<CounterModel>(
  builder: (context, counter, child) {
    return Column(
      children: [
        Text('Count: ${counter.count}'),
        ElevatedButton(
          onPressed: counter.increment,
          child: Text('Increment'),
        ),
      ],
    );
  },
)
```

setState() vs Provider

setState(): simple and local state management.

Provider: useful when state is shared by multiple widgets.

setState(): state normally belongs to one StatefulWidget.

Provider: separates state/model logic from UI and allows dependent widgets to listen.

Choose the simplest approach that fits the application.

Experiment 5(b) – Lab Exercise

Part A: Create a counter application using StatefulWidget and setState().

Part B: Create the same counter application using Provider.

Use ChangeNotifier to store the counter value.

Call notifyListeners() after changing the value.

Use Consumer to display the updated value.

Compare the two implementations.

Learning Outcomes

Understand Row, Column and Stack layout structures.

Use mainAxisAlignment and crossAxisAlignment appropriately.

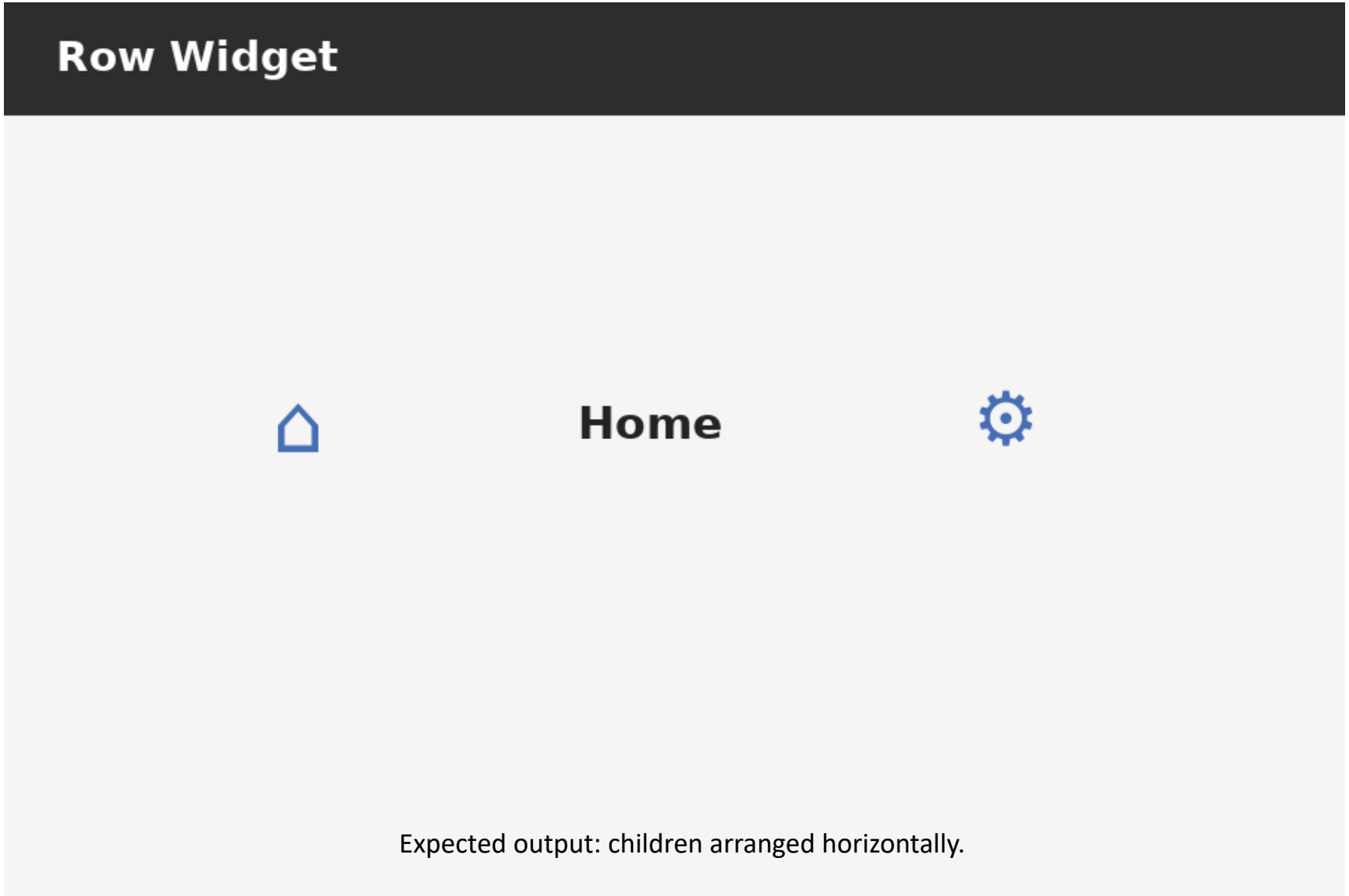
Combine multiple layout widgets to build a UI.

Understand state and why state management is required.

Implement local state using setState().

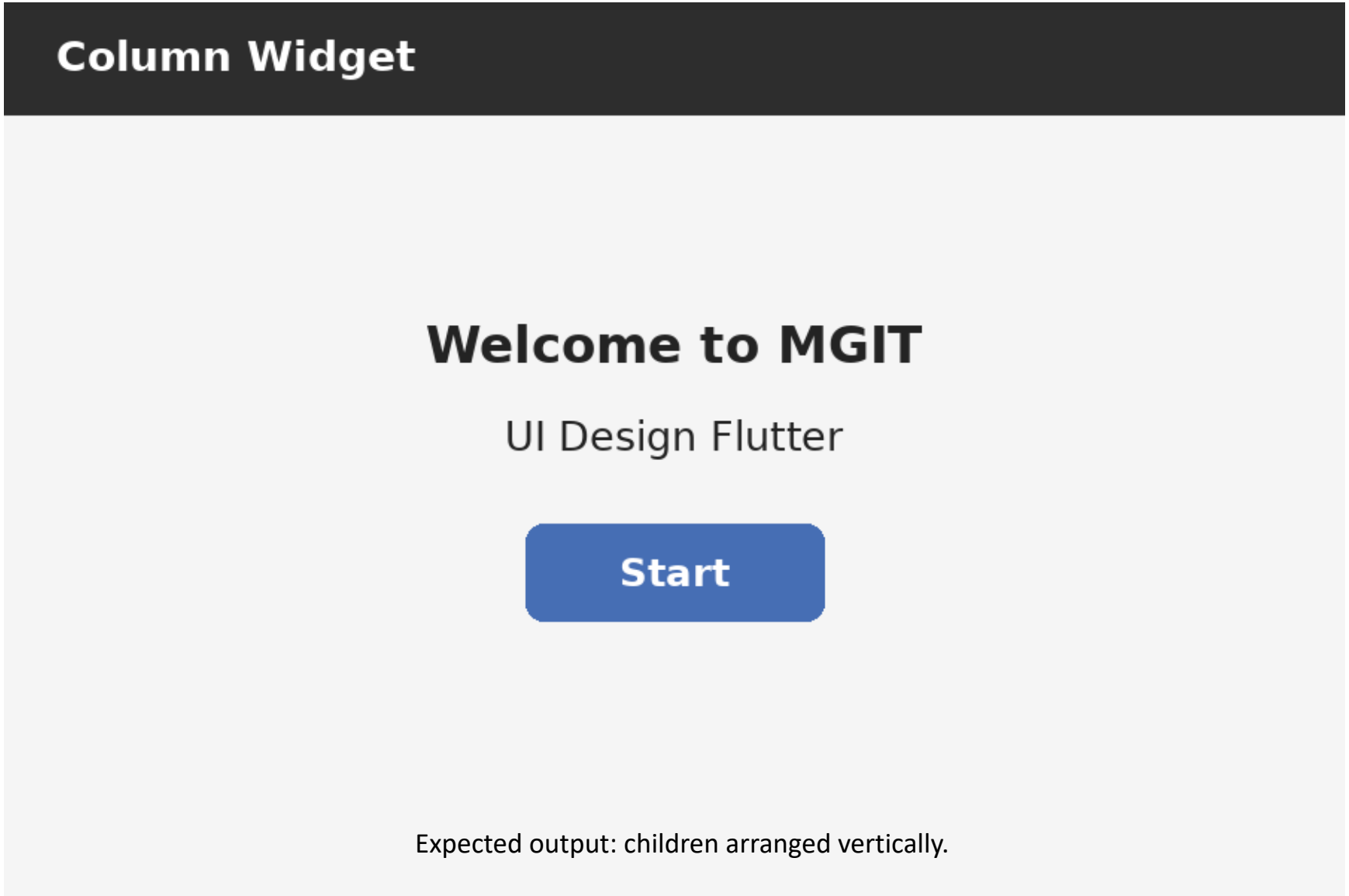
Implement shared state using Provider and ChangeNotifier.

Output – Row Widget



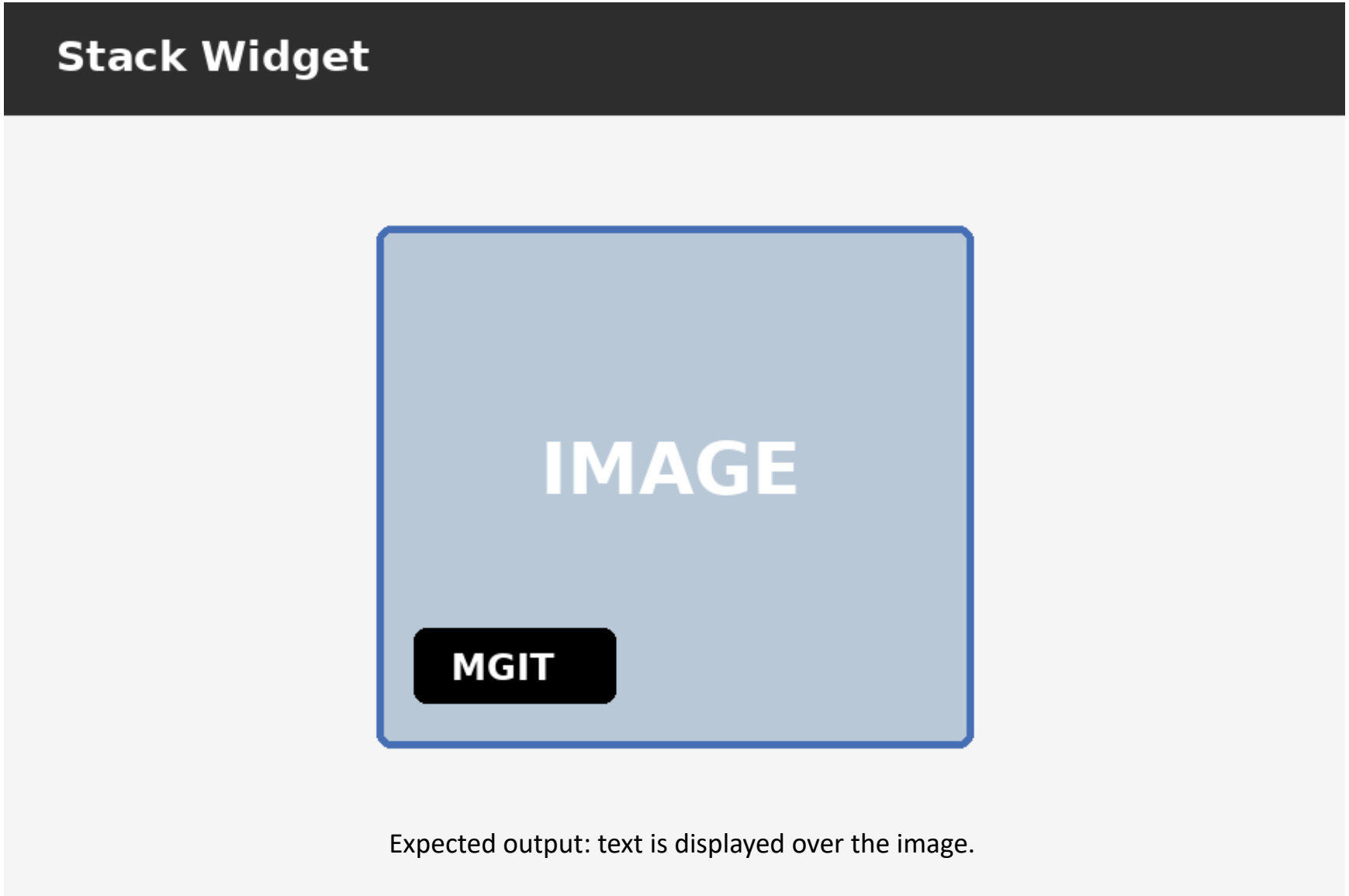
Expected output: children arranged horizontally.

Output – Column Widget



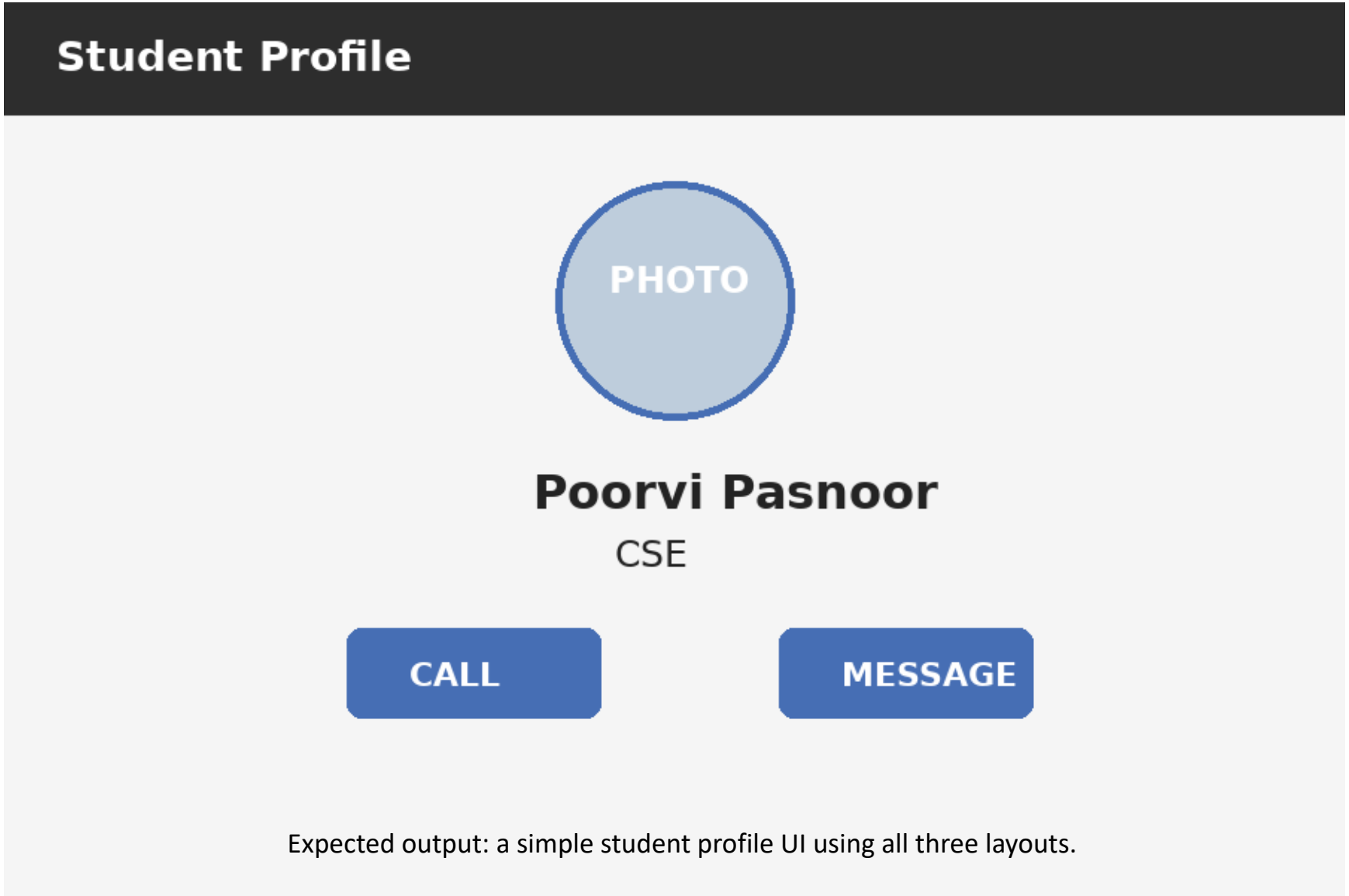
Expected output: children arranged vertically.

Output – Stack Widget



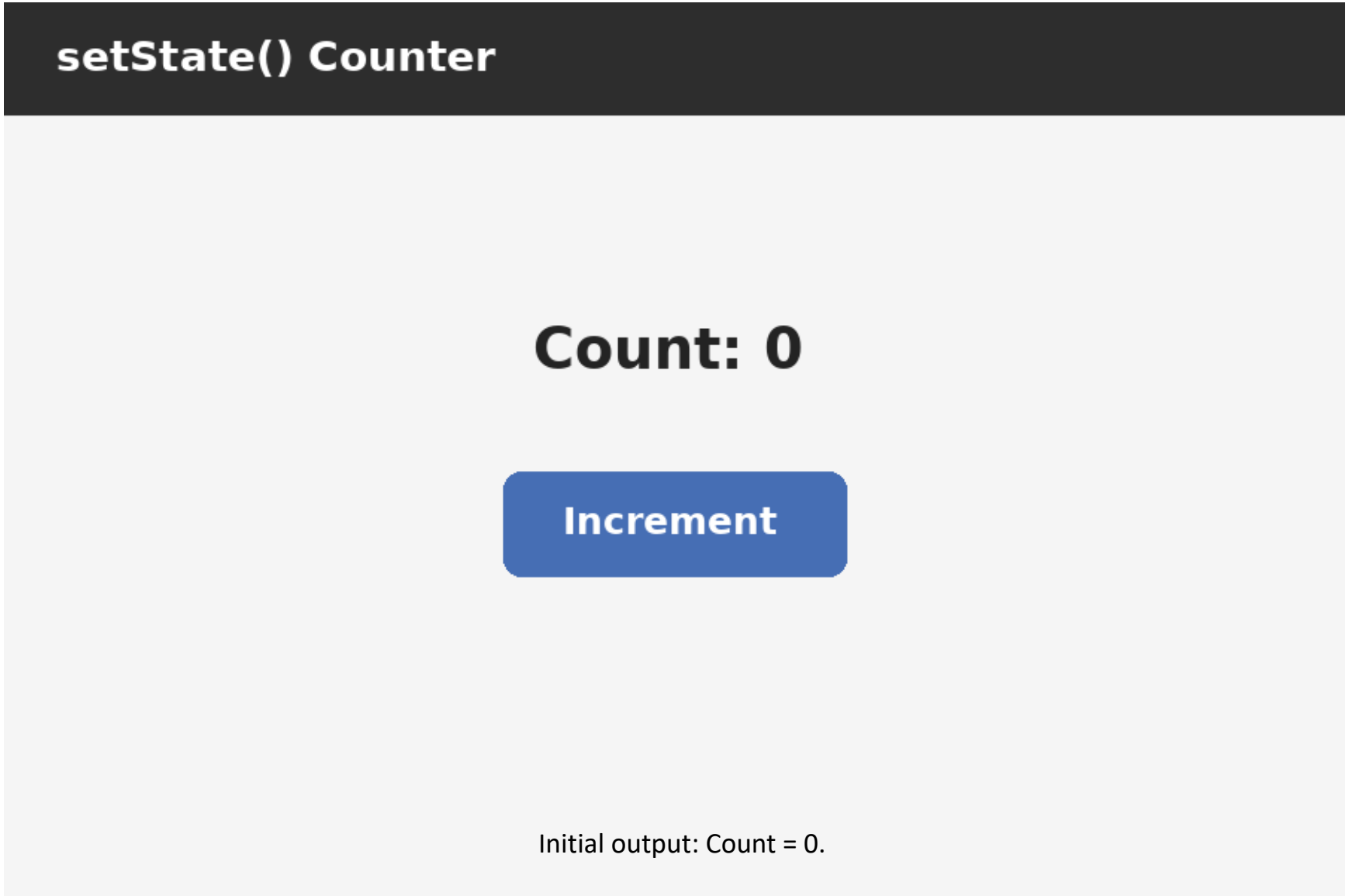
Expected output: text is displayed over the image.

Output – Combined Row + Column + Stack



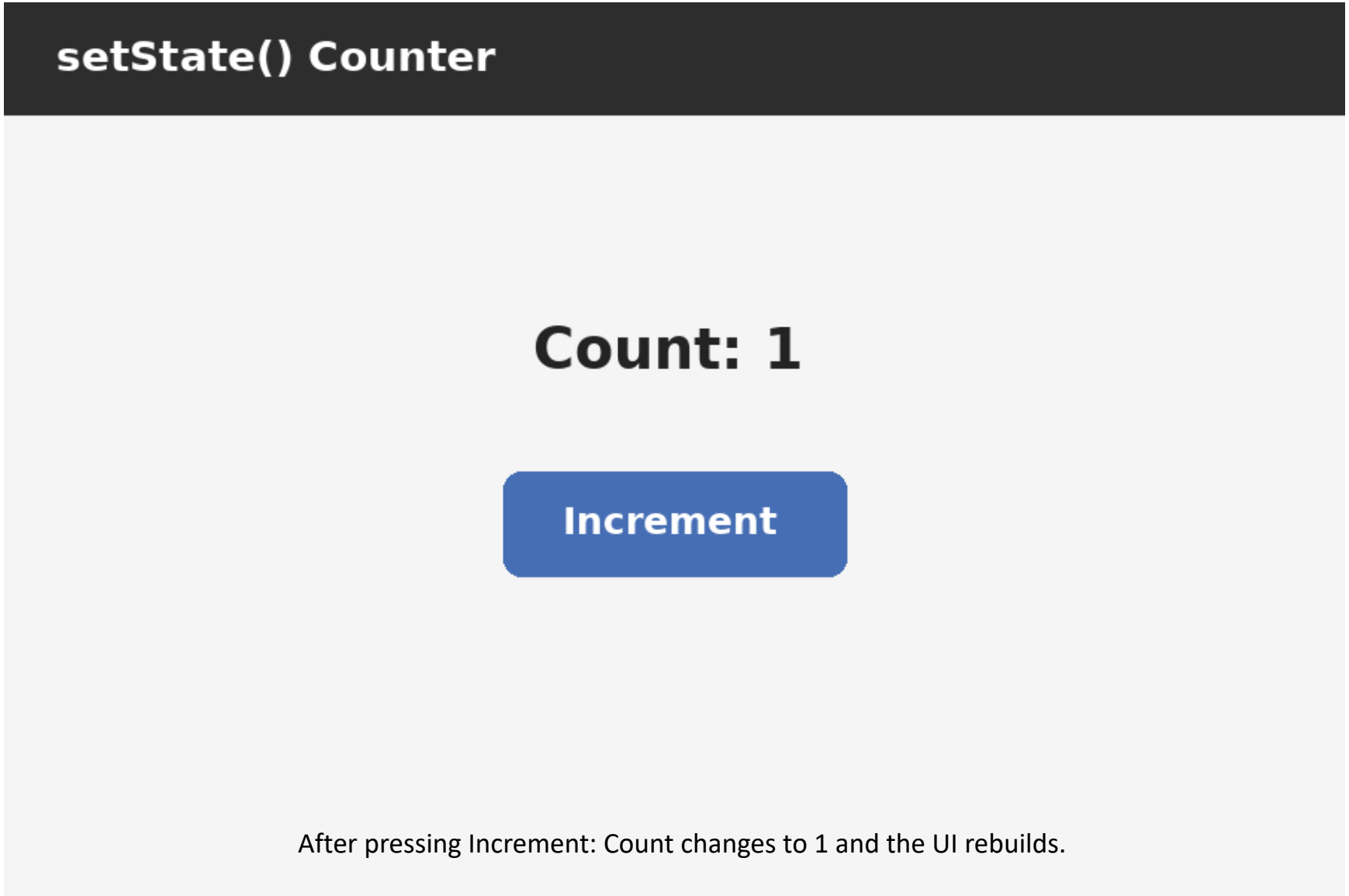
Expected output: a simple student profile UI using all three layouts.

Output – setState() Counter (Initial)



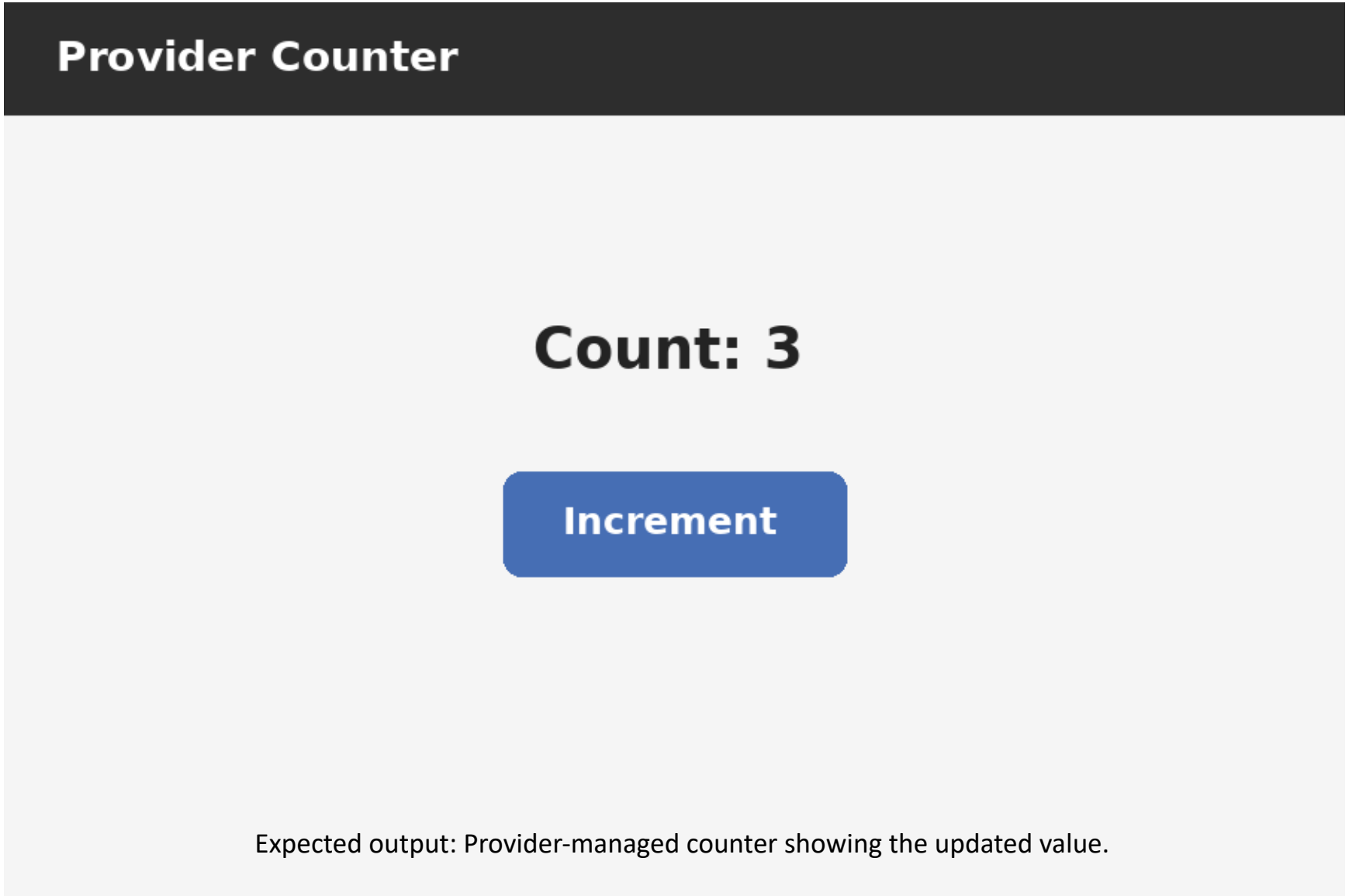
Initial output: Count = 0.

Output – setState() Counter (After Click)



After pressing Increment: Count changes to 1 and the UI rebuilds.

Output – Provider Counter



Expected output: Provider-managed counter showing the updated value.

Step-by-Step Instructions

- **Step 1**: Add your image to project folderInside your Flutter project directory:
- Navigate to:
- your_project_name/assets/images/
- If assets or images folders do not exist, create them.

```
your_project/
└── assets/
    └── images/
        └── bg.jpg     ← (your background image here)
```

Step 2: Update pubspec.yaml file

- Open **pubspec.yaml** and **add** this **under flutter: section**:

```
flutter:
  assets:
    - assets/images/bg.jpg
```

```
flutter:
  assets:
    - assets/images/bg.jpg
```

Make sure:

- Indentation is correct (use **2 spaces**).
- The file is saved after editing.
- **Get dependencies** – clicl-its update pub file